



Build innovative software applications that leverage the Databricks platform and agentic AI to solve real-world problems for a wearable application with streaming data. Create an AI agent that analyzes data and provide



An end-to-end architecture leveraging the **Databricks Platform**, **Agentic AI**, a **React Frontend**, and continuous streaming telemetry from a **Bluetooth Low Energy (BLE) wearable** (like the Polar H10) enables real-time physiological intelligence. Databricks

1. Streaming Infrastructure & Databricks Medallion Architecture



Ingestion Gateway: The mobile app captures raw BLE heart rate (HR) and RR-intervals (milliseconds between beats) from the Polar H10 at up to 1000 Hz. The mobile app batches packets into 1-second JSON payloads and streams them to **AWS Kinesis** or **Apache Kafka**.

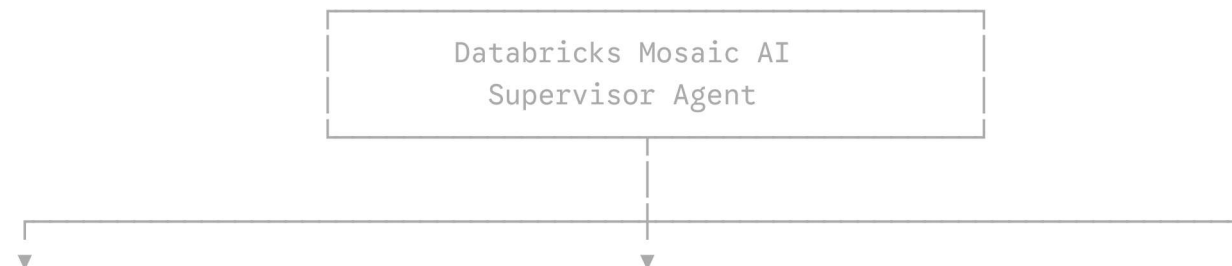
Bronze Delta Table (Raw Ingestion): Databricks Lakeflow Pipelines / Delta Live Tables (DLT) consume the stream into append-only streaming tables, preserving raw microsecond-level timestamps and sensor quality metrics.

Silver Delta Table (Cleansed Metrics): DLT stateful streaming with watermarks computes rolling aggregations: average heart rate, beat-to-beat variability (RMSSD/SDNN for HRV), and cardiac noise filtering.

Gold Delta Table (Feature Store & Baselines): Calculates daily baselines (resting heart rate, recovery efficiency, training load strain) and persists them into the **Databricks Feature Store** and **Vector Search** for RAG.

2. Agentic AI System Architecture

The AI Agent acts as an autonomous physiological analyst using the **Databricks Mosaic AI Agent Framework**. Rather than just answering static questions, the agent uses tools, APIs, and multi-step reasoning to orchestrate data analysis. Databricks + 1



```
Unity Catalog Tool  
get_hr_baselines()
```

```
Unity Catalog Tool  
detect_anomalies()
```

```
Unity Catalog Tool  
generate_chart_spec()
```

Registered Tools (Unity Catalog & MCP Protocol)

1. `get_user_hr_baselines(user_id, timeframe)` : Queries Gold Delta tables via Databricks Genie / SQL endpoints for historical 14-day trends.
2. `detect_hrv_anomalies(user_id, stream_window)` : Invokes an Isolation Forest or Autoencoder model hosted on **Databricks Model Serving** to check for unexpected autonomic nervous system strain. Databricks
3. `generate_chart_spec(metric_type, time_range)` : Generates structured JSON (Vega-Lite or Recharts specification) for UI chart rendering.
4. `execute_third_party_action(action, payload)` : Connects via Model Context Protocol (MCP) to third-party APIs (e.g., Apple Health, TrainingPeaks, or pushing calendar adjustments).

Multi-Step Agentic Workflow Example

1. **Trigger:** The Silver pipeline detects a sustained 15% spike in resting HR combined with a drop in HRV during sleep.
2. **Autonomous Querying:** The agent calls `get_user_hr_baselines()` to check the 14-day average and retrieves yesterday's workout intensity logs.
3. **Root-Cause Reasoning:** The agent identifies an overtraining signature (high strain + suppressed parasympathetic activity).

4. **Artifact Generation:** It invokes `generate_chart_spec()` to construct a dual-axis chart showing the HRV drop relative to acute workload.
5. **Action Execution:** The agent issues a push notification to the user and suggests adjusting the current day's high-intensity workout to active recovery.

3. Actionable Insights & Visualizations

Key Actionable Insights

Cardiovascular Recovery Score: "Your resting heart rate is elevated by 8 bpm above baseline, and sleeping HRV dropped by 22 ms. Your autonomic nervous system is under strain."

Action: "Replace today's interval run with a 30-minute Zone 1 recovery walk."

Aerobic Drift Detection: "During your 45-minute base run, your heart rate drifted upward by 14% while pace remained constant, signaling dehydration or glycogen depletion."

Action: "Increase fluid/electrolyte intake by 250 ml on runs longer than 40 minutes."

Acute Stress/Anomaly Alert: "Unexplained HR spike (112 bpm) detected at 2:15 AM during deep sleep cycle."

Action: "Flagged for sleep hygiene analysis. Correlate with late-night meals or alcohol intake."

Generated Diagrams

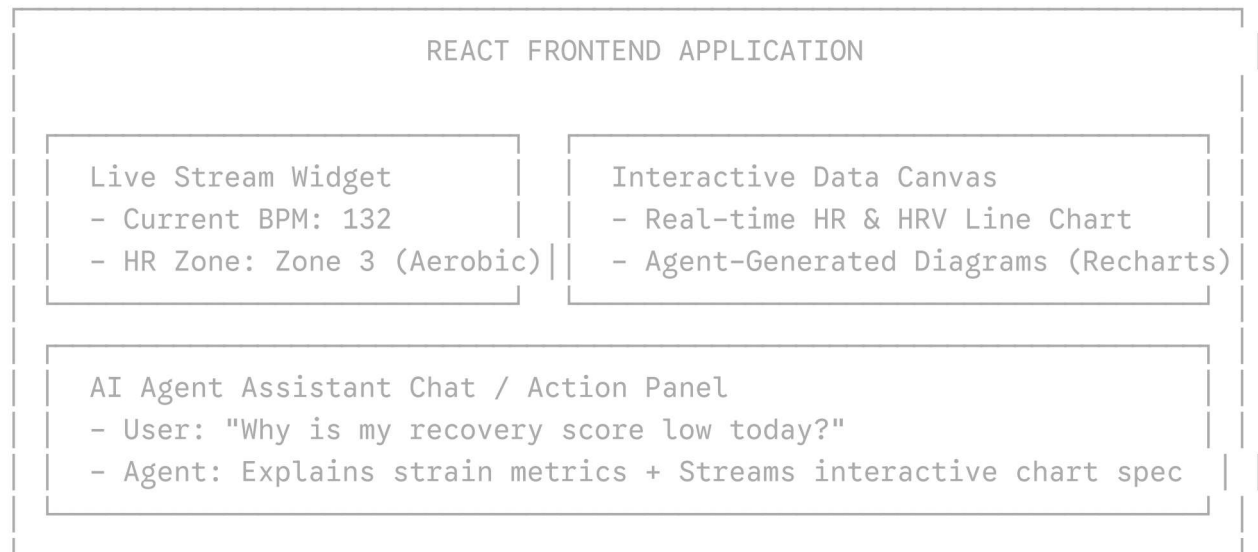
HR Zone Distribution: Real-time donut/stacked-bar chart showing time spent in Zone 1 (Recovery) through Zone 5 (Anaerobic).

HRV Strain Matrix: Dynamic scatter plot showing 7-day cumulative strain versus parasympathetic recovery status.

Anomaly Timeline: Time-series line chart with shaded confidence intervals and interactive callout flags at anomaly points.

4. React Front-End Application Architecture

The React app serves as an interactive intelligence dashboard where users view live streaming data and collaborate with the AI agent.



Streaming Transport: Uses Server-Sent Events (SSE) or WebSockets to connect React components directly to Databricks Model Serving / Gateway endpoints for sub-second

visual updates.

Dynamic UI Rendering (`Recharts` / `Nivo`): Rather than returning static image charts, the AI Agent emits raw JSON chart specs. A custom React component (`<DynamicChart spec="{agentChartSpec}"/>`) renders responsive, interactive charts on the fly.

Interactive Chart Exploration: Clicking on an anomaly marker on the chart feeds the time range back into the agent conversation prompt (*"Analyze what happened between 14:00 and 14:30"*).

Would you like to dive deeper into the code implementation for the Databricks DLT streaming pipeline or the React dynamic chart component?

Yes, give me all the information i need before i start building

Here is the complete implementation guide, API specifications, and codebase required to build the solution before development begins.

1. Device Communication & Ingestion Payload

The Polar H10 exposes standard GATT Heart Rate Service (`0x180D`) and Heart Rate Measurement characteristic (`0x2A37`). The BLE payload contains flags, Heart Rate (BPM), and RR-intervals (milliseconds between heartbeats).

BLE JavaScript/TypeScript Connection (Mobile Gateway / Web App)

JavaScript



```
// Web Bluetooth API Gateway to read Polar H10 BLE characteristics
async function connectPolarH10(onDataReceived) {
  const device = await navigator.bluetooth.requestDevice({
    filters: [{ services: ['heart_rate'] }]
  });

  const server = await device.gatt.connect();
  const service = await server.getPrimaryService('heart_rate');
  const characteristic = await service.getCharacteristic('heart_rate_measurement');

  characteristic.startNotifications();
  characteristic.addEventListener('characteristicvaluechanged', (event) => {
    const value = event.target.value;
    const flags = value.getUint8(0);
    const is16Bit = flags & 0x01;
    let hr = is16Bit ? value.getUint16(1, true) : value.getUint8(1);

    // Parse RR-intervals if present (bit 4 of flags)
    const rrIntervals = [];
    let offset = is16Bit ? 3 : 2;
    if (flags & 0x10) {
      while (offset < value.byteLength) {
        const rr = value.getUint16(offset, true) / 1024.0 * 1000.0; // convert to ms
        rrIntervals.push(Math.round(rr));
        offset += 2;
      }
    }

    onDataReceived({
      user_id: "usr_99812",
```

```
        device_id: device.id,  
        timestamp: new Date().toISOString(),  
        heart_rate: hr,  
        rr_intervals_ms: rrIntervals  
    });  
});  
}
```

JSON Stream Schema (Injected into AWS Kinesis or Apache Kafka)

JSON



```
{  
  "user_id": "usr_99812",  
  "device_id": "polar_h10_a18b",  
  "timestamp": "2026-10-03T12:54:37.102Z",  
  "heart_rate": 142,  
  "rr_intervals_ms": [421, 418, 425]  
}
```

2. Databricks Lakehouse Pipeline (Delta Live Tables)

Create a Databricks DLT pipeline (`dlt_wearable_pipeline.py`) to process raw streams into cleansed HRV metrics using standard Heart Rate Variability math:

$$RMSSD = \sqrt{\frac{1}{N-1} \sum_{i=1}^{N-1} (RR_{i+1} - RR_i)^2}$$

Python



```
import dlt
from pyspark.sql.functions import *
from pyspark.sql.types import *

# 1. Bronze Layer: Streaming Ingestion from Event Hubs / Kinesis
@dlt.table(
    name="wearable_bronze",
    comment="Raw streaming BLE telemetry from wearable devices"
)
def wearable_bronze():
    return (
        spark.readStream
            .format("kinesis")
            .option("streamName", "telemetry-hr-stream")
            .option("initialPosition", "latest")
            .option("region", "us-east-1")
            .load()
            .select(from_json(col("data").cast("string"), SchemaMatchingStruct).alias("payload"))
            .select("payload.*")
    )

# 2. Silver Layer: 1-Minute Stateful Aggregations and HRV Processing
@dlt.table(
```

```

        name="wearable_silver",
        comment="Aggregated 1-minute HR and computed RMSSD HRV metrics"
    )
@dlt.expect_or_drop("valid_heart_rate", "avg_hr > 30 AND avg_hr < 230")
def wearable_silver():
    return (
        dlt.read_stream("wearable_bronze")
        .withWatermark("timestamp", "2 minutes")
        .groupBy(
            col("user_id"),
            window(col("timestamp"), "1 minute")
        )
        .agg(
            avg("heart_rate").alias("avg_hr"),
            max("heart_rate").alias("max_hr"),
            min("heart_rate").alias("min_hr"),
            flatten(collect_list("rr_intervals_ms")).alias("all_rr_ms")
        )
        # UDF or PySpark expression to compute RMSSD from all_rr_ms list
        .select(
            col("user_id"),
            col("window.start").alias("window_start"),
            col("window.end").alias("window_end"),
            round(col("avg_hr"), 1).alias("avg_hr"),
            col("max_hr"),
            col("min_hr")
        )
    )

# 3. Gold Layer: Baseline Feature Store Table
@dlt.table(
    name="wearable_gold_baselines",
    comment="14-day rolling baseline heart rate and autonomic strain metrics"
)

```

```
def wearable_gold_baselines():  
    return (  
        dlt.read("wearable_silver")  
        .groupBy("user_id")  
        .agg(  
            avg("avg_hr").alias("baseline_hr_14d"),  
            stddev("avg_hr").alias("std_hr_14d")  
        )  
    )
```

3. Unity Catalog Tools & Databricks Mosaic AI Agent

Register Unity Catalog functions to grant the AI agent controlled access to data and ML models.

Databricks Community

Unity Catalog Python Tools

SQL



```
-- Register baseline retriever tool in Unity Catalog  
CREATE OR REPLACE FUNCTION main.default.get_user_baselines(  
    user_id STRING COMMENT 'Unique identifier for the target user'  
)  
RETURNS TABLE (baseline_hr_14d DOUBLE, std_hr_14d DOUBLE)  
LANGUAGE SQL  
COMMENT 'Fetches historical 14-day baseline metrics for heart rate from Gold tables'  
AS RETURN  
    SELECT baseline_hr_14d, std_hr_14d
```

```
FROM main.default.wearable_gold_baselines
WHERE user_id = get_user_baselines.user_id;
```

Python



```
# Unity Catalog Python Tool for dynamic Recharts specification generation
import json

def generate_recharts_spec(metric_type: str, time_window: str) -> str:
    """Generates a JSON configuration for rendering Recharts components on frontend.

    Args:
        metric_type: 'heart_rate' or 'hrv'
        time_window: '1h', '24h', or '7d'
    """
    chart_config = {
        "chartType": "LineChart",
        "xAxisKey": "timestamp",
        "series": [
            {"dataKey": "avg_hr", "stroke": "#ef4444", "name": "Heart Rate (BPM)"},
            {"dataKey": "baseline_hr", "stroke": "#9ca3af", "name": "14D Baseline"},
        ],
        "tooltip": True,
        "legend": True
    }
    return json.dumps(chart_config)
```

Agent Orchestrator Definition (agent.py)

Python



```
import ChatDatabricks, UCFunctionToolkit
from create_react_agent

model = ChatDatabricks(
    model="databricks-meta-llama-3-3-70b-instruct")

function_names=[
    "baselines",
    "echarts_spec"

]

persona_and_output_contracts = """
You are an expert physiological agent analyzing streaming wearable data.
Your tasks are:
- Detect anomalies or user recovery:
- Suggest available tools.
- Provide training or recovery steps.
- Generate `echarts_spec` when visual context is beneficial, returning the chart spec as a JSON block."""

def create_agent(
    llm, tools, state_modifier=system_prompt)

def deploy_agent(
    agent, tools, state_modifier=system_prompt)
```

```
:",
  "latabricks-langchain", "langgraph"]
```

4. React Application & Dynamic Renderer

The front end consumes live server-sent events for data streaming and renders dynamic charts directly from JSON specifications output by the agent.

Dynamic Chart Component (AgentChart.jsx)

JavaScript



```
import React from 'react';
import { LineChart, Line, XAxis, YAxis, Tooltip, CartesianGrid, Legend, ResponsiveContainer } from 'recharts';

export const AgentChart = ({ spec, data }) => {
  if (!spec) return null;
  const config = typeof spec === 'string' ? JSON.parse(spec) : spec;

  return (
    <div className="w-full h-64 bg-slate-900 p-4 rounded-xl shadow-md border border-slate-800">
      <ResponsiveContainer width="100%" height="100%">
        <LineChart data={data}>
          <CartesianGrid strokeDasharray="3 3" stroke="#334155" />
          <XAxis dataKey={config.xAxisKey} stroke="#94a3b8" />
          <YAxis stroke="#94a3b8" />
          {config.tooltip && <Tooltip contentStyle={{ backgroundColor: '#1e293b', border: '1px solid #334155' }} />}
        </LineChart>
      </ResponsiveContainer>
    </div>
  );
};
```

```

    {config.legend && <Legend />}
    {config.series.map((s, idx) => (
      <Line
        key={idx}
        type="monotone"
        dataKey={s.dataKey}
        stroke={s.stroke}
        name={s.name}
        strokeDasharray={s.strokeDasharray || "0"}
        dot={false}
      />
    ))}
  </LineChart>
</ResponsiveContainer>
</div>
);
};

```

Agent Chat & Visual Dashboard (Dashboard.jsx)

JavaScript



```

import React, { useState, useEffect } from 'react';
import { AgentChart } from './AgentChart';

export const Dashboard = ({ userId }) => {
  const [telemetry, setTelemetry] = useState([]);
  const [currentHR, setCurrentHR] = useState(null);
  const [messages, setMessages] = useState([]);

```

```

const [inputPrompt, setInputPrompt] = useState('');
const [activeChartSpec, setActiveChartSpec] = useState(null);

// 1. WebSockets / SSE connection for streaming telemetry
useEffect(() => {
  const eventSource = new EventSource(`/api/stream/telemetry?user_id=${userId}`);
  eventSource.onmessage = (e) => {
    const payload = JSON.parse(e.data);
    setCurrentHR(payload.heart_rate);
    setTelemetry((prev) => [...prev.slice(-30), payload]); // keep last 30 readings
  };
  return () => eventSource.close();
}, [userId]);

// 2. Query Databricks Agent Model Serving Endpoint
const handleQueryAgent = async () => {
  const userMsg = { sender: 'user', text: inputPrompt };
  setMessages((prev) => [...prev, userMsg]);
  setInputPrompt('');

  const res = await fetch('/serving-endpoints/wearable-agent/invocations', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      messages: [{ role: 'user', content: inputPrompt }],
      custom_inputs: { user_id: userId }
    })
  });

  const data = await res.json();
  const botReply = data.choices[0].message.content;

  // Check if agent returned a chart spec block
  const specMatch = botReply.match(/```json_chart\n([\s\S]*?)\n```/);

```

```

    if (specMatch) {
      setActiveChartSpec(JSON.parse(specMatch[1]));
    }

    setMessages((prev) => [...prev, { sender: 'agent', text: botReply }]);
  };

  return (
    <div className="grid grid-cols-12 gap-4 p-6 bg-slate-950 min-h-screen text-slate-
      { /* Real-time Telemetry Widget */ }
      <div className="col-span-4 bg-slate-900 p-6 rounded-2xl border border-slate-800
        <h2 className="text-xl font-bold mb-2">Live BLE Telemetry</h2>
        <div className="text-6xl font-black text-red-500 mb-4">
          {currentHR ? `${currentHR} BPM` : 'Connecting...'}
        </div>
        <AgentChart spec={activeChartSpec} data={telemetry} />
      </div>

      { /* Agent Interaction Panel */ }
      <div className="col-span-8 bg-slate-900 p-6 rounded-2xl border border-slate-800
        <div className="overflow-y-auto h-96 space-y-4 mb-4">
          {messages.map((m, idx) => (
            <div key={idx} className={`p-3 rounded-lg ${m.sender === 'user' ? 'bg-bl
              {m.text}
            </div>
          ))}
        </div>
        <div className="flex gap-2">
          <input
            className="flex-1 bg-slate-800 border border-slate-700 rounded-lg px-4 p
            value={inputPrompt}
            onChange={(e) => setInputPrompt(e.target.value)}
            placeholder="Ask agent: Why is my resting HR high today?"
          />

```

```
        <button onClick={handleQueryAgent} className="bg-red-600 px-6 py-2 rounded-
      </div>
    </div>
  </div>
);
};
```

5. Development Pre-flight Checklist

Service / Tool	Configuration Requirement
Databricks Workspace	Serverless SQL Warehouses enabled, Unity Catalog enabled, and Delta Live Tables entitlement.
Model Serving	Serve LLMs (e.g., databricks-meta-llama-3-3-70b-instruct) or custom LangGraph models.
Streaming Broker	AWS Kinesis Stream or Azure Event Hubs instance configured with ingestion SAS/IAM keys.
Client Host	HTTPS/SSL localhost certificate (Web Bluetooth API is blocked by browsers over unencrypted HTTP).
Python Libraries	databricks-sdk , databricks-langchain , pyspark , mlflow >= 2.12 .

Service / Tool

Configuration Requirement

Node.js Libraries`recharts , lucide-react , tailwindcss .`

Are you deploying on AWS or Az
Hubs) will you connect to DLT?

deploy with AWS and with AWS Kinesis for the event bus

1. Provision AWS Kinesis Data Stream & IAM Roles

Create Kinesis Data Stream

Run the AWS CLI to create a provisioned stream with 2 shards for high throughput (handles ~2,000 records/sec write capacity).

Bash



```
aws kinesis create-stream \  
  --stream-name telemetry-hr-stream \  
  --shard-count 2 \  
  --region us-east-1
```

Ingestion IAM Policy (`kinesis-writer-policy.json`)

Attach this policy to the Cognito identity pool or IAM user that authenticates the client mobile/web app gateway sending data to Kinesis.

JSON



```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "kinesis:PutRecord",
        "kinesis:PutRecords"
      ],
      "Resource": "arn:aws:kinesis:us-east-1:AWS_ACCOUNT_ID:stream/telemetry-hr-strea"
    }
  ]
}
```



Databricks Streaming Reader IAM Policy (

databricks-kinesis-reader-policy.json)

Attach this policy to the AWS IAM Role associated with your Databricks Cluster / Instance Profile to grant Databricks read access to Kinesis.

JSON



```
{
  "Version": "2012-10-17",
```

```

    "Statement": [
      {
        "Effect": "Allow",
        "Action": [
          "kinesis:GetRecords",
          "kinesis:GetShardIterator",
          "kinesis:DescribeStream",
          "kinesis:DescribeStreamSummary",
          "kinesis:ListShards",
          "kinesis:ListStreams"
        ],
        "Resource": "arn:aws:kinesis:us-east-1:AWS_ACCOUNT_ID:stream/telemetry-hr-stre"
      }
    ]
  }
}

```

2. Ingestion Gateway (Client Node.js / AWS SDK v3)

This script bridges the BLE stream from the client device directly to AWS Kinesis.

TypeScript



```

import { KinesisClient, PutRecordCommand } from "@aws-sdk/client-kinesis";

// Initialize AWS Kinesis Client
const kinesisClient = new KinesisClient({
  region: "us-east-1",
  credentials: {

```

```

:essKeyId: process.env.REACT_APP_AWS_ACCESS_KEY_ID!,
:retAccessKey: process.env.REACT_APP_AWS_SECRET_ACCESS_KEY!

    async function sendTelemetryToKinesis(payload: {
      _id: string;
      _e_id: string;
      _stamp: string;
      _rate: number;
      _intervals_ms: number[];

      : command = new PutRecordCommand({
        :eamName: "telemetry-hr-stream",
        :titionKey: payload.user_id, // Route user records to specific shard
        :a: Buffer.from(JSON.stringify(payload))

      :
    })
    const response = await kinesisClient.send(command);
    console.log("Telemetry ingested to Kinesis, SequenceNumber:", response.SequenceNumber);
    catch (error) {
      console.error("Error pushing telemetry to Kinesis:", error);
    }
  }
}

```

3. Databricks Kinesis DLT Pipeline Setup

DLT Bronze Pipeline Definition (dlt_kinesis_bronze.py)

In Databricks, store AWS credentials securely using **Databricks Secrets** or rely on the cluster IAM instance profile.

Python



```
import dlt
from pyspark.sql.functions import col, from_json, expr
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, ArrayType

# Define expected JSON Schema
telemetry_schema = StructType([
    StructField("user_id", StringType(), True),
    StructField("device_id", StringType(), True),
    StructField("timestamp", StringType(), True),
    StructField("heart_rate", IntegerType(), True),
    StructField("rr_intervals_ms", ArrayType(IntegerType()), True)
])

@dlt.table(
    name="wearable_bronze",
    comment="Raw streaming BLE telemetry read directly from AWS Kinesis"
)
def wearable_bronze():
    return (
        spark.readStream
        .format("kinesis")
        .option("streamName", "telemetry-hr-stream")
        .option("region", "us-east-1")
        .option("initialPosition", "latest")
        # Optional: Use Databricks Secret Manager if not using Instance Profiles
        .option("awsAccessKey", dbutils.secrets.get(scope="aws-scope", key="aws-acce:"))
```

```

.option("awsSecretKey", dbutils.secrets.get(scope="aws-scope", key="aws-secret-key"))
.load()
# Data payload arrives as binary in the 'data' column
.select(from_json(col("data").cast("string"), telemetry_schema).alias("payload"))
.select("payload.*")
.withColumn("timestamp", col("timestamp").cast(TimestampType()))
)

```

4. Deploying Agent Endpoint on Databricks Model Serving

Deploy the agent to Databricks Model Serving with auto-scaling backing on AWS infrastructure.

Python



```

from databricks.sdk import WorkspaceClient
from databricks.sdk.service.serving import EndpointCoreConfigInput, ServedEntityInput

w = WorkspaceClient()

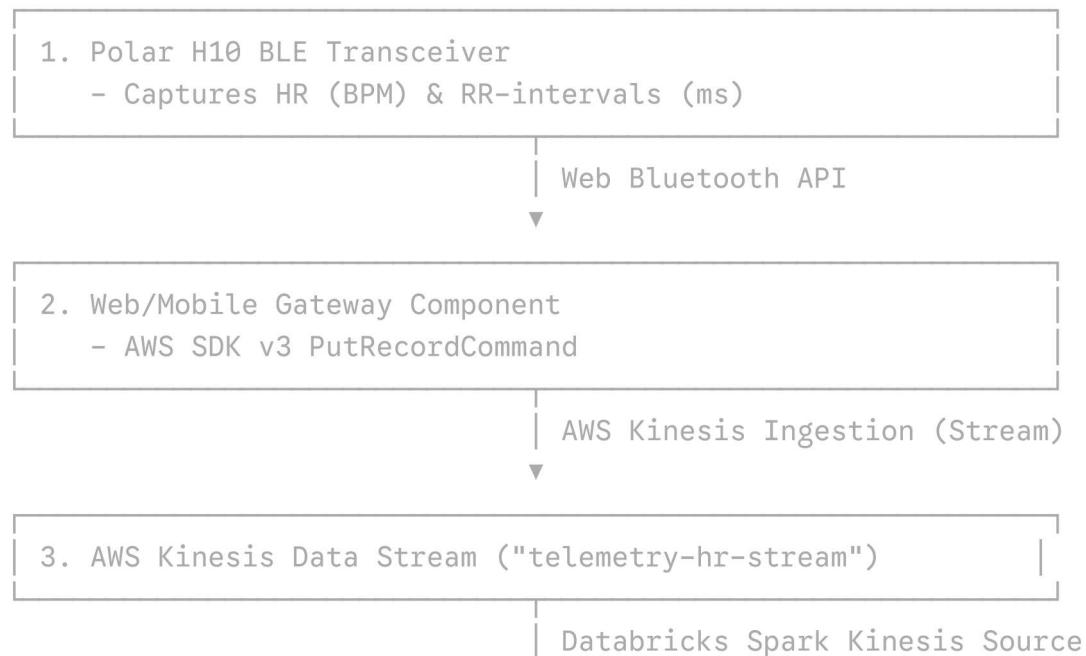
# Create or update Databricks Model Serving Endpoint
endpoint_name = "wearable-agent-endpoint"

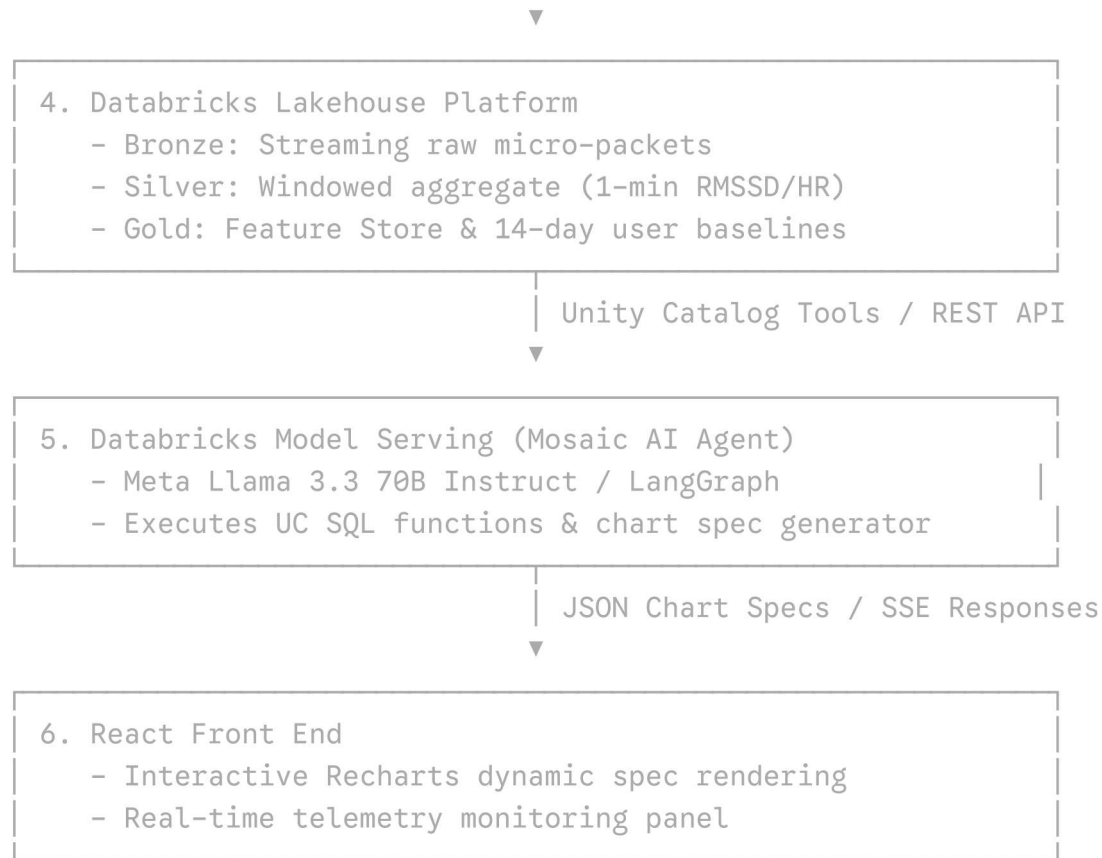
w.serving_endpoints.create_and_wait(
    name=endpoint_name,
    config=EndpointCoreConfigInput(
        served_entities=[
            ServedEntityInput(
                entity_name="main.default.wearable_agent_model",

```

```
entity_version="1",  
workload_size="Small", # Scalable AWS EC2 instance pool under the hood  
scale_to_zero_enabled=False  
)  
]  
)  
)  
  
print(f"Endpoint '{endpoint_name}' successfully deployed on Databricks AWS infrastructure")
```

5. End-to-End System Pipeline Summary





Quick Commands for Deployment Execution

1. Initialize Stream:

```
aws kinesis create-stream --stream-name telemetry-hr-stream --shard-count 2 --region us-east-1
```

2. Deploy DLT Pipeline: Import `dlt_kinesis_bronze.py` into Databricks Workflows and set target dataset to `main.default` .

3. **Register Tools & Deploy Agent:** Execute `agent.py` in a Databricks Notebook to register